

Tests Unitaires et Validation d'Infrastructure avec Pester

Ce document constitue le guide de référence méthodologique et technique sur l'industrialisation des tests automatisés sous PowerShell avec **Pester**. Il détaille la démarche BDD (*Behavior-Driven Development*), l'organisation hiérarchique des scénarios de test, le catalogue des assertions Should, le mécanisme de simulation de composants critiques (**Mocking**), l'audit des fonctions internes d'un module (InModuleScope), ainsi que la mesure de couverture de code dans un pipeline d'automatisation.

1. Test en Automatisation d'Infrastructure

En administration système, la phase de recette d'un script est souvent artisanale :

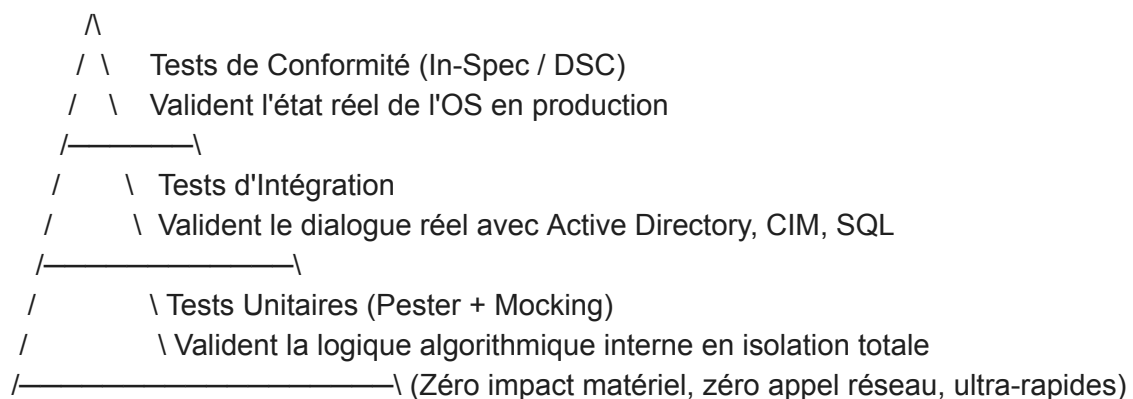
- On exécute le script "à la main" sur un serveur de test ou directement sur son poste.
- On constate visuellement si une commande s'est bien déroulée.
- Dès qu'une modification mineure est apportée au code, on réexécute l'outil sans vérifier l'ensemble des cas limites (*edge cases*).

1.1 Le paradoxe du test d'infrastructure

Tester du code d'exploitation présente un risque systémique que le développement applicatif standard ne connaît pas :

- Un script d'infrastructure interagit directement avec le noyau de l'OS, l'annuaire Active Directory, les services de messagerie, les volumes de stockage ou les hyperviseurs.
- Tenter d'exécuter un script pour "voir s'il marche" peut provoquer la suppression irréversible de comptes utilisateurs, le formatage accidentel d'une partition ou le redémarrage intempestif d'un cluster en production.

1.2 La pyramide de validation en entreprise



Un test unitaire ne doit **jamais** dépendre de l'état d'un serveur physique ni d'une connexion réseau. Il valide uniquement que, pour des entrées données, le code prend les bonnes décisions logiques et appelle les bonnes commandes avec les bons paramètres.

2. Présentation du Framework Pester

Pester est le framework officiel de test unitaire et d'automatisation de recette pour PowerShell. Écrit nativement en PowerShell, il est maintenu par la communauté et soutenu par Microsoft.

2.1 Évolution des versions : Pester v4 vs Pester v5

Critère d'évaluation	Pester v4 (Historique / Inclus PS 5.1)	Pester v5 (Standard Industriel Moderne)
Modèle d'exécution	Exécution linéaire immédiate (au fil de l'eau).	Découpage strict en deux phases : Discovery (recensement) puis Run (exécution).
Portée des variables	Variables globales partagées entre blocs (effets de bord).	Cloisonnement strict des portées au sein des blocs parent/enfant.
Cycle de vie	Blocs d'initialisation artisanaux.	Blocs formels BeforeAll, BeforeEach, AfterEach, AfterAll.
Vitesse d'exécution	Standard.	Optimisée pour les suites massives et les pipelines CI/CD.

Note d'environnement Windows PowerShell 5.1 :

Bien que Windows 10 et Windows Server intègrent par défaut une très ancienne révision (Pester 3.4.0), la mise à niveau vers **Pester 5** s'effectue simplement via PowerShell Gallery :

```
Install-Module -Name Pester -Force -SkipPublisherCheck -Scope CurrentUser
```

3. L'Architecture Hiérarchique BDD : Describe,

Context, It

Pester adopte la démarche **BDD** (*Behavior-Driven Development* ou Développement Piloté par le Comportement). Les tests ne sont pas rédigés comme du code machine, mais comme un cahier des charges lisible décrivant le comportement attendu du composant.

3.1 Anatomie structurelle d'un fichier .Tests.ps1

```
Describe "Get-InfraDisque" {  
    Context "Lorsque le lecteur C: dispose d'un espace abondant (> 15 %)" {  
        It "Doit renvoyer un statut 'OPTIMAL'" {  
            # Code de validation  
        }  
  
        It "Ne doit pas déclencher d'écriture dans le journal d'alerte" {  
            # Code de validation  
        }  
    }  
  
    Context "Lorsque le lecteur C: est saturé (< 15 %)" {  
        It "Doit renvoyer un statut 'CRITIQUE'" {  
            # Code de validation  
        }  
  
        It "Doit consigner un événement d'avertissement dans le log" {  
            # Code de validation  
        }  
    }  
}
```

3.2 Rôle de chaque niveau de la hiérarchie

- **Describe (Le Sujet)** : Délimite le composant global testé (le script ou la fonction avancée). C'est le conteneur racine.
- **Context (Le Scénario / La Situation)** : Établit l'état de départ du système (préconditions). Il permet de grouper les tests selon des contextes opérationnels distincts (ex: cas nominal, panne réseau, compte désactivé, seuil dépassé).
- **It (L'Assertion / Le Comportement attendu)** : Représente l'unité atomique de test. Il commence conventionnellement par un verbe d'action (« *Doit renvoyer...* », « *Doit lever une erreur...* », « *Ne doit pas modifier...* »). Si une exception est levée à l'intérieur du bloc It, le test échoue (Failed). Si le code arrive à son terme sans erreur, le test réussit

(Passed).

4. Le Moteur d'Assertions : L'Opérateur Should

À l'intérieur des blocs It, la validation de conformité s'effectue via l'instruction **Should**.

Si la condition n'est pas vérifiée, Should lève une exception descriptive capturée par Pester pour détailler l'échec.

4.1 Catalogue des assertions indispensables

A. Égalité et Typage

Égalité insensible à la casse

\$Resultat.Statut | Should -Be "OPTIMAL"

Égalité stricte sensible à la casse

\$CodeSite | Should -BeExactly "PARIS-01"

Inégalité

\$Processus.Id | Should -Not -Be 0

Contrôle du type .NET

\$Resultat | Should -BeOfType [PSCustomObject]

\$DateControle | Should -BeOfType [System.DateTime]

B. Valeurs Nulles, Booléens et Collections

Présence ou absence de données

\$ObjetRetourne | Should -Not -BeNullOrEmpty

\$Erreur | Should -BeNullOrEmpty

Vérité logique

\$Resultat.Ouvert | Should -BeTrue

\$FichierExiste | Should -BeFalse

Comptage d'éléments dans une collection

\$ServeursAudit | Should -HaveCount 5

C. Comparaisons Numériques et Chaînes

Bornes et grandeurs

```
$DisqueLibreGo | Should -BeGreaterThan 10
$TempsExecution | Should -BeLessThanOrEqual 30
```

```
# Recherche de motifs textuels (Wildcard / Regex)
$CheminFichier | Should -Match '^[A-Z]:\\'
$MessageLog | Should -Like "*CRITIQUE"
```

D. Interception d'Erreurs (-Throw)

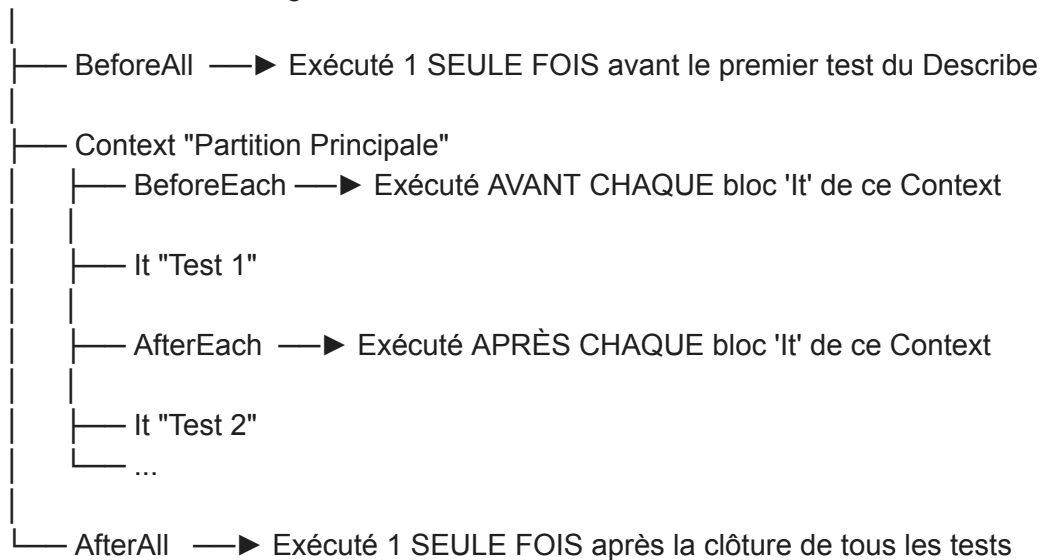
Pour tester qu'une fonction réagit sagement et bloque l'exécution sur une entrée corrompue :

```
It "Doit refuser un seuil négatif" {
    # L'appel doit impérativement être encapsulé dans un ScriptBlock { }
    { Get-InfraDisque -Lecteur "C" -SeuilAlertePct -5 } | Should -Throw
}
```

5. Le Cycle de Vie des Données de Test : Les Blocs de Configuration (*Fixtures*)

Pour éviter de dupliquer la préparation des données d'un test à l'autre, Pester fournit des blocs de synchronisation du cycle de vie :

Describe "Audit Stockage"



Exemple de manipulation de données éphémères :

```
Describe "Gestionnaire de Logs" {  
  BeforeAll {  
    $DossierTest = Join-Path -Path $TestDrive -ChildPath "LogsTest"  
    New-Item -Path $DossierTest -ItemType Directory -Force | Out-Null  
  }  
  
  It "Doit écrire un fichier horodaté" {  
    $FichierLog = Join-Path -Path $DossierTest -ChildPath "audit.log"  
    "Donnée" | Out-File -FilePath $FichierLog  
    Test-Path $FichierLog | Should -BeTrue  
  }  
  
  AfterAll {  
    Remove-Item -Path $DossierTest -Recurse -Force -ErrorAction SilentlyContinue  
  }  
}
```

Le lecteur magique \$TestDrive :

Pester crée automatiquement un PSDrive temporaire et isolé nommé \$TestDrive:\ (ou accessible via la variable \$TestDrive). Tout fichier créé dans cet espace est **automatiquement purgé de votre machine** à la fin de l'exécution du bloc Describe. C'est le bac à sable idéal pour tester la création de fichiers et dossiers sans polluer le disque dur physique.

6. L'Art du Mocking (Mock) : Tester sans Détruire

Le **Mocking** est la compétence la plus importante en ingénierie de test d'infrastructure.

6.1 Pourquoi « mocker » ?

Si votre fonction doit redémarrer un serveur (Restart-Computer), supprimer des fichiers temporaires (Remove-Item) ou interroger un contrôleur Active Directory (Get-ADUser) :

- Vous ne voulez pas redémarrer votre poste de travail pendant la série de tests.
- Vous ne disposez pas forcément d'un serveur Active Directory sur votre machine locale de développement.
- Vous souhaitez simuler un cas de panne (ex: simuler que le disque dur est plein ou qu'une requête HTTP 500 se produit) sans devoir provoquer une vraie panne sur le système.

Un **Mock** intercepte l'appel à une commande native et lui substitue un comportement factice

entièrement maîtrisé.

6.2 Syntaxe de base d'un Mock

```
Describe "Test de Nettoyage" {  
  It "Doit tenter de supprimer le fichier temporaire" {  
    # Interception de la cmdlet Remove-Item : elle n'exécutera rien sur le disque  
    Mock Remove-Item { return $true }  
  
    # Exécution de notre fonction métier  
    Invoke-PurgeDossier -Chemin "C:\Temp\fichiers.tmp"  
  
    # Vérification formelle : Est-ce que Remove-Item a été appelée exactement 1 fois ?  
    Assert-MockCalled Remove-Item -Times 1 -Exactly  
  }  
}
```

6.3 Mocks Conditionnels (-ParameterFilter)

En production, une fonction peut appeler plusieurs fois la même cmdlet pour des cibles différentes. Le paramètre -ParameterFilter permet de définir des réponses sur mesure selon les arguments transmis :

```
Describe "Get-InfraDisque avec CIM factice" {  
  It "Doit renvoyer un statut critique si le disque C: a moins de 15% d'espace libre" {  
    # Simulation chirurgicale de Get-CimInstance :  
    # Ne s'active QUE si l'interrogation cible la classe Win32_LogicalDisk  
    Mock Get-CimInstance -ParameterFilter { $ClassName -eq 'Win32_LogicalDisk' }  
    -MockWith {  
      [PSCustomObject]@{  
        DeviceID = 'C:'  
        Size     = 100GB  
        FreeSpace = 10GB # 10 % d'espace libre (sous le seuil de 15 %)  
        DriveType = 3  
      }  
    }  
  
    # Appel de la fonction audité  
    $Bilan = Get-InfraDisque -Lecteur "C"  
  
    # Assertions  
    $Bilan.Statut | Should -Be "CRITIQUE"
```

```

    $Bilan.LibrePct | Should -Be 10.0
  }
}

```

6.4 Vérification des Appels (Assert-MockCalled)

Tester le résultat d'une fonction ne suffit pas : il faut prouver qu'elle a déclenché les bonnes actions techniques en arrière-plan.

```

# Vérifier qu'une commande dangereuse n'a JAMAIS été appelée
Assert-MockCalled Stop-Process -Times 0 -Exactly

```

```

# Vérifier qu'une alerte a été émise au moins une fois
Assert-MockCalled Write-JournalModule -Times 1 -AtLeast

```

```

# Vérifier qu'une commande a été appelée avec des arguments précis
Assert-MockCalled Set-Service -ParameterFilter { $Name -eq 'Spooler' -and $Status -eq
'Running' }

```

7. Tester un Module d'Entreprise : Isolation et Fonctions Privées

Dans un module d'entreprise conforme au pattern **Public / Private**, les fonctions utilitaires internes (Private*.ps1) ne sont pas exportées dans la session.

Comment tester une fonction privée ou mocker une commande appelée à l'intérieur d'un module scellé ?

7.1 L'instruction maîtresse : InModuleScope

L'instruction `InModuleScope <NomDuModule> { ... }` force Pester à pénétrer directement dans le **scope de session interne du module** (*Module Session State*).

```

Describe "Tests des Fonctions Internes du Module InfraTools" {
  # On force l'exécution à l'intérieur de la frontière privée du module
  InModuleScope "InfraTools" {
    It "La fonction privée Convert-TailleOctet doit convertir 1 Go avec exactitude" {
      # Convert-TailleOctet est invisible à l'extérieur,
      # mais parfaitement accessible ici !
      $Taille = Convert-TailleOctet -Octets 1073741824 -Unite "GB"
    }
  }
}

```

```

        $Taille | Should -Be 1.0
    }

    It "Doit mocker une commande interne appelée par une fonction privée" {
        Mock Add-Content { }
        Write-JournalModule -Message "Test unitaire" -Niveau "INFO"
        Assert-MockCalled Add-Content -Times 1
    }
}
}

```

8. Exécution, Restitution et Métriques de Couverture

8.1 Exécuter des tests avec Invoke-Pester

Dans la console PowerShell, on exécute la suite de tests via Invoke-Pester :

```
# Exécuter tous les tests du dossier courant
Invoke-Pester
```

```
# Cibler un fichier précis en mode verbeux (Détail de chaque test)
Invoke-Pester -Path ".\Tests\Get-InfraDisque.Tests.ps1" -Output Detailed
```

8.2 Restitution visuelle console

Running tests from 'C:\Projets\InfraTools\Tests\Get-InfraDisque.Tests.ps1'

Describing Get-InfraDisque

Context Lorsque le lecteur C: est sain (> 15%)

[+] Doit retourner un objet [PSCustomObject] valide 45ms

[+] Doit indiquer le statut OPTIMAL 12ms

Context Lorsque le lecteur C: est saturé (< 15%)

[+] Doit indiquer le statut CRITIQUE 14ms

[+] Doit consigner une entrée d'avertissement dans le journal 18ms

Tests completed in 1.15s

Tests Passed: 4, Failed: 0, Skipped: 0, Total: 4

8.3 Analyse de la Couverture de Code (Code Coverage)

La couverture de code mesure le pourcentage d'instructions de votre code source réellement parcourues et exécutées lors de la campagne de test :

$$\text{Taux de Couverture (\%)} = \left(\frac{\text{Nombre de lignes exécutées par les tests}}{\text{Nombre total de lignes de code analysées}} \right) \times 100$$

Pour activer l'analyse de couverture avec Pester :

```
$Config = New-PesterConfiguration
$Config.Run.Path = ".\Tests"
$Config.CodeCoverage.Enabled = $true
$Config.CodeCoverage.Path = ".\Public\*.ps1", ".\Private\*.ps1"
```

```
Invoke-Pester -Configuration $Config
```

Pester relève alors les lignes oubliées (ex: un bloc catch jamais déclenché ou une branche else non évaluée). En entreprise, un standard d'ingénierie logiciel impose généralement une couverture minimale de 80 % .

8.4 Intégration Continue (CI/CD) et Export XML

Dans un pipeline GitLab CI, Azure DevOps ou GitHub Actions, Pester exporte ses résultats au format standard international **NUnit** :

```
$Config = New-PesterConfiguration
$Config.TestResult.Enabled = $true
$Config.TestResult.OutputFormat = "NUnitXml"
$Config.TestResult.OutputPath = ".\artifacts\test-results.xml"
```

```
Invoke-Pester -Configuration $Config
```

L'orchestrateur CI/CD lit ce fichier XML pour valider le commit ou bloquer le déploiement en cas d'échec.

9. Synthèse des Bonnes Pratiques de Test

1. **Un fichier de test par fonction publique** : Adopter la convention stricte de nommage `<NomDeFonction>.Tests.ps1` placée dans un répertoire `Tests/`.

2. **Ne jamais faire d'accès réel dans un test unitaire** : Tout appel à Get-CimInstance, Test-NetConnection, Invoke-RestMethod, Get-Service, au registre ou au disque physique doit être **intercepté par un Mock**.
3. **Tester les scénarios nominaux ET les scénarios d'échec** : Un bon testeur ne valide pas seulement ce qui marche. Il s'assure qu'une mauvaise entrée déclenche bien une erreur (Should -Throw) et que les timeouts réseau sont interceptés proprement par les blocs catch.
4. **Isoler les tests (TestDrive)** : Pour les tests nécessitant de lire ou générer de vrais fichiers d'épreuve, manipuler exclusivement le disque virtuel \$TestDrive:\.
5. **Vérifier les effets de bord avec Assert-MockCalled** : Valider qu'une fonction de lecture (Get-*) n'appelle jamais une commande de modification (Set-*, Remove-*, Restart-*).